Prelucrarea sirurilor de valori

Definiție

- Un șir de numere este o înșiruire de numere sau de valori pe care le notăm $\mathbf{a_1, a_2, a_3}$ ... în care $\mathbf{a_1, a_2, a_3}$ sunt termenii șirului, iar **1, 2, 3** reprezintă pozițiile pe care le ocupă termenii șirului.
- Notăm ($\mathbf{a_n}$) cu $\mathbf{n \in N^*}$ șirul care are termenul general $\mathbf{a_n}$.
- Numim rang al unui termen din șir poziția pe care o ocupă termenul în șir.

Un șir de numere se poate defini prin:

- prin enumerarea termenilor șirului, atunci când acesta este un șir finit

Exemplu: **2, 5, 17, -1, 19, 20, 35, -2, -10, 101, 15**

- cu ajutorul unei formule sau proprietăți ale termenilor șirului

Exemplu: șirul primelor 10 numere impare naturale: **1, 3, 5, 7, 9, 11, 13, 15, 17, 19**

- printr-o relație între elementele șirului:

Exemplu: șirul lui Fibonacci: **0, 1, 1, 2, 3, 5, 8, 13**, ….

În funcție de modul în care este definit șirul de valori, prelucrările asupra termenilor șirului se pot grupa în două categorii:

- Prelucrări specifice șirurilor finite de valori, pentru care termenii sunt dați prin enumerarea valorilor termenilor. Problemele cu șiruri finite de valori au două ipoteze posibile: șirul are un număr cunoscut de termeni sau un număr necunoscut de termeni și o condiție de continuare/oprire a citirii.
    - citirea valorilor din șir;
    - parcurgerea șirului și prelucrarea elementelor;
    - afișarea rezultatelor prelucrării.
- Prelucrări specifice șirurilor în care fiecare termen are o anumită proprietate, sau se află într-o anumită relație cu alți termeni ai șirului, sau este determinat de o formulă matematică. Atunci când șirul este definit prin descrierea elementelor mulțimii termenilor trebuie să deducem formula matematică care definește fiecare termen al șirului.
    - determinarea formulei termenului general,
    - generarea termenilor șirului.
- Calcularea valorii maxime, valorii minime, a sumelor valorilor și numărarea valorilor sunt prelucrări specifice șirurilor finite de valori, pentru care termenii sunt dați prin enumerarea valorilor termenilor.
- **Algoritmul pentru calcularea valorii minime**
- Pentru calcularea valorii minime se va utiliza o variabilă care va reține valoarea minimă astfel:
- **Pasul 1**: variabila va reține prima valoarea din șirul de valori sau cea mai mare valoare din domeniul de valori din care fac parte termenii șirului

- **Pasul 2**: se parcurge șirul de valori și se compară variabila cu fiecare termen al șirului. În situația în care un termen al șirului are o valoare mai mică decât valoarea reținută în variabilă atunci se variabila va reține valoarea termenului respectiv.

```
if(minim>termen_curent)
{
    minim=termen_curent;
}
```

Exemplu:

Fişierul **numere.in** conţine mai multe numere întregi (cel mult $\mathbf{10^6}$), fiecare număr având cel mult **9** cifre. Se cere să se afişeze pe ecran, cea mai mică valoare pară memorată în fişier. Dacă în fişier nu se află cel puțin o valoare pară pe ecran se va afișa mesajul **nu exista**.

Rezolvare:

Pasul 1: Se inițializează o variabilă **minim** cu valoarea **1000000** (cea mai mare valoare din domeniul de valori) și o altă variabilă, **gasit** cu valoarea **0**

Pasul 2: Se citesc valorile din fișier și se verifică dacă sunt valori pare și sunt mai mici decât valoare reținută în variabila **minim**, atunci se rețin în variabila minim și valoarea variabilei găsit devine **1**.

Pasul 3: Dacă variabila **gasit** are valoarea **1** se va afișa variabila **minim**, altfel se va afișa mesajul **nu exista**

Limbajul C++

```
#include <iostream>
#include<fstream>
using namespace std;
ifstream fin("numere.in");
int main()
{
    int val,minim,gasit=0;
    minim=1000000;
    while(fin>>val)
    {
        if(val%2==0 && minim>val)
        {
            minim=val;
            gasit=1;
        }
    }
    if(gasit==1)
    {
        cout<<minim;
    }
    else
    {
        cout<<"nu exista";
    }
    return 0;
}
```

Limbajul C

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fin=fopen("numere.in","r");
    int val,minim,gasit=0;
    minim=1000000;
    while(fscanf(fin,"%d",&val)!=EOF)
    {
        if(val%2==0 && minim>val)
        {
            minim=val;
            gasit=1;
        }
    }
    if(gasit==1)
    {
        printf("%d\n",minim);
    }
    else
    {
        printf("nu exista\n");
    }
    return 0;
}
```

Algoritmul pentru calcularea valorii maxime

Pentru calcularea valorii minime se va utiliza o variabilă care va reține valoarea maximă astfel:

Pasul 1: variabila va reține prima valoarea din șirul de valori sau cea mai mică valoare din domeniul de valori din care fac parte termenii șirului

Pasul 2: se parcurge șirul de valori și se compară variabila cu fiecare termen al șirului. În situația în care un termen al șirului are o valoare mai mare decât valoarea reținută în variabilă atunci se variabila va reține valoarea termenului respectiv.

```c
if (maxim<termenul_curent)
    {
        maxim=termenul_curent;
    }
```

Exemplu:

Fişierul **numere.in** conţine mai multe numere întregi (cel mult $\mathbf{10^6}$), fiecare număr având cel mult **9** cifre. Se cere să se afişeze pe ecran, cea mai mare valoare pară memorată în fişier. Dacă în fişier nu se află cel puţin o valoare pară pe ecran se va afişa mesajul **nu exista**.

Rezolvare:

Pasul 1: Se inițializează o variabilă **maxim** cu valoarea **-1000000** (cea mai mare valoare din domeniul de valori) și o altă variabilă, **gasit** cu valoarea **0**

Pasul 2: Se citesc valorile din fișier și se verifică dacă sunt valori pare și sunt mai mari decât valoare reținută în variabila **maxin**, atunci se rețin în variabila maxim și valoarea variabilei găsit devine **1**.

Pasul 3: Dacă variabila **gasit** are valoarea **1** se va afișa variabila **maxim**, altfel se va afișa mesajul **nu exista**

Limbajul C++

```
#include <iostream>
#include<fstream>
using namespace std;
ifstream fin("numere.in");
int main()
{
    int val,maxim,gasit=0;
    maxim=-1000000;
    while(fin>>val)
    {
        if(val%2==0 && maxim<val)
        {
            maxim=val;
            gasit=1;
        }
    }
    if(gasit==1)
    {
        cout<<maxim;
    }
    else
    {
        cout<<"nu exista";
    }
    return 0;
}
```

Limbajul C

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fin=fopen("numere.in","r");
    int val,maxim,gasit=0;
    maxim=-1000000;
    while(fscanf(fin,"%d",&val)!=EOF)
    {
        if(val%2==0 && maxim<val)
        {
            maxim=val;
            gasit=1;
        }
    }
    if(gasit==1)
    {
        printf("%d\n",maxim);
    }
    else
    {
        printf("nu exista\n");
    }
    return 0;
}
```

Exemplu: Numarul de aparitii ale valorii maxime:

```
#include <iostream>
using namespace std;
int main()
{
    int n,i, max=0, nrap,x;
    cin>>n;
    cin>>max;
    nrap=1;
    for(i=2; i<=n; i++)
    {
        cin>>x;
        if(x>max)
        {
            max=x;
            nrap=1;
        }
        else if(max==x)
            nrap++;
    }
    cout<<max<<" "<<nrap;
    return 0;
}
```

Vecinii unui termen al unui şir

Fie şirul **51 25 102 47 43 61 50 38 220 114 116 4**

Primul și ultimul termen din șir au doar un singur vecin: primul termen are un vecin aflat în dreapta și ultimul termen are un vecin aflat în stânga. Ceilalți termeni au câte doi vecini, unul se află în stânga, celălalt în dreapta termenului.

Enunț: Pentru un şir de numere naturale, un termen din şir care are doi vecini, termenul precedent şi termenul următor din şir, numiți vecinii termenului.
Fişierul **numere.in** conţine un şir de cel mult $\mathbf{10^6}$ de numere naturale având maximum **6** cifre fiecare, numere separate prin câte un spaţiu. Scrieţi un program C/C++ care citeşte toate numerele din fişier şi afişează numărul de termeni ai şirului care sunt strict mai mari decât cei doi vecini ai săi.
Exemplu: dacă fişierul **numere.in** are următorul conţinut: 51 **201** 100 43 46 **61** 50 31 20 114 **116** 4 se va afișa valoarea **3**

Rezolvare: Se vor citi primii doi termeni din fișier în două variabile **a** și **b**. Începând cu cel de al treilea termen citirea se va realiza în variabila **c**. Se va compara variabila **b** cu variabilele **a** și **c**, iar dacă **b** conține 0 valoare mai mare aceasta se va număra.

```
#include<iostream>
#include<fstream>
using namespace std;
ifstream fin("numere.in");
int main()
{
    int a,b,c, gasit=0;
    fin>>a>>b;
    while(fin>>c)
    {
        if(b>a && b>c)
        {
            gasit++;
        }
        a=b;
        b=c;
    }
    if(gasit==0)
    {
        cout<<"Nu exista";
    }
    else
    {
        cout<<gasit;
    }
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fin=fopen("numere.in","r");
    int a,b,c, gasit=0;
    fscanf(fin,"%d %d",&a,&b);
    while(fscanf(fin,"%d",&c)!=EOF)
    {
        if(b>a && b>c)
        {
            gasit++;
        }
        a=b;
        b=c;
    }
    if(gasit==0)
    {
        printf("Nu exista");
    }
    else
    {
        printf("%d",gasit);
    }
    return 0;
}
```

Subșir de valori

- Un **subșir** al unui șir de valori este compus dintr- o parte din termenii unui șir care își păstrează ordinea inițială în șir.
- Un șir finit de valori cu **n** elemente are $\mathbf{2^n}$ subșiruri.

Exemplu: Fie șirul de valori numerice: **2, 5, 17, -1, 19, 20, 35, -2, -10, 101, 15**

- fie subșirul compus din toți termenii pari: **2, 20, -2, -10**,
- fie subșirul compus din termenii impari: **5, 17, -1, 19, 35, 101, 15**

Enunț: Fişierul **numere.in** conţine, pe o singură linie, cel mult $\mathbf{10^6}$ de numere naturale nenule de cel mult **9** cifre fiecare, numerele fiind separate prin câte un spaţiu. Scrieţi un program C/C++ care citeşte toate numerele din fişierul **numere.in** și afișează pe ecran, separate prin câte un spaţiu, toate numerele pare
în ordinea în care acestea apar în fişier. Dacă fişierul **numere.in** nu conţine niciun număr par, atunci se va afișa mesajul **nu exista**.
Exemplu: dacă fişierul **numere.in** are următorul conţinut: **2 3 1 4 7 2 5 8 6** atunci se va afișa pe ecran **2 4 2 8 6**

Rezolvare: Se vor citi valorile din fișier și se vor afișa valorile pare. Se va utiliza o variabilă **gasit** care se inițializează cu valoarea **0**. La afișarea unei valori pare variabila **gasit** va lua valoarea **1**.

```
#include<iostream>
#include<fstream>
using namespace std;
ifstream fin("numere.in");
int main()
{
    int val, gasit=0;
    while(fin>>val)
    {
        if(val%2==0)
        {
            cout<<val<<" ";
            gasit=1;
        }

    }
    if(gasit==0)
    {
        cout<<"nu exista";
    }
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fin=fopen("numere.in","r");
    int val, gasit=0;
    while(fscanf(fin,"%d",&val)!=EOF)
    {
        if(val%2==0)
        {
            printf("%d ",val);
            gasit=1;
        }
    }
    if(gasit==0)
    {
        printf("nu exista");
    }

    return 0;
}
```

Enunț: Pe prima linie a fişierului numere.in se află un număr natural nenul **n** (**n≤10^6**), iar pe a doua linie a fişierului se află un şir format din **n** numere naturale, despărţite prin câte un spaţiu, fiecare număr fiind format din cel mult **9** cifre. Scrieţi un program C/C++ care citeşte valorile din fişier şi afişează pe ecran mesajul **DA** dacă toate elementele impare din şir sunt în ordine strict crescătoare şi mesajul **NU** în caz contrar.

Exemplu: dacă fișierul **numere.in** are conținutul:

7
10 ***11*** **12** ***33*** **42 1354** ***421 677***

se va afișa pe ecran mesajul: **DA**

Rezolvare Se va utiliza o variabilă **ant** care se inițializează cu valoarea -1. Se citesc valorile din fișier în variabila **val** și valorile impare se compară cu valoarea anterioară impară reținută în variabila **ant**.

```cpp
#include<iostream>
#include<fstream>
using namespace std;
ifstream fin("numere.in");
int main()
{
    int n, val, gasit=0, ant=-1;
    fin>>n;
    for (int i=1; i<=n; i++)
    {
        if(val%2==1)
        {
            if(val>ant)
            {
                ant=val;
            }
            else
            {
                gasit=1;
            }
        }
    }
    if(gasit==0)
    {
        cout<<"DA";
    }
    else
    {
        cout<<"NU";
    }
    return 0;
}
```

```cpp
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fin=fopen("numere.in","r");
    int n,val, gasit=0,ant=-1;
    fscanf(fin,"%d",&n);
    for(int i=1; i<=n; i++)
    {
        fscanf(fin,"%d",&val);
        if(val%2==1)
        {
            if(val>ant)
            {
                ant=val;
            }
            else
            {
                gasit=1;
            }
        }
    }
    if(gasit==0)
    {
        printf("DA");
    }
    else
    {
        printf("NU");
    }
    return 0;
}
```